

ASSIGNMENT-11.2

NAME: DUGYALA VINAY

HTNO: 2303A51050

BATCH: 16

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

- Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

- Ask AI to locate repeated computations within the program.
- Encapsulate repeated logic into reusable functions.
- Verify that program output remains unchanged after refactoring.
- Add proper docstrings to all defined functions.

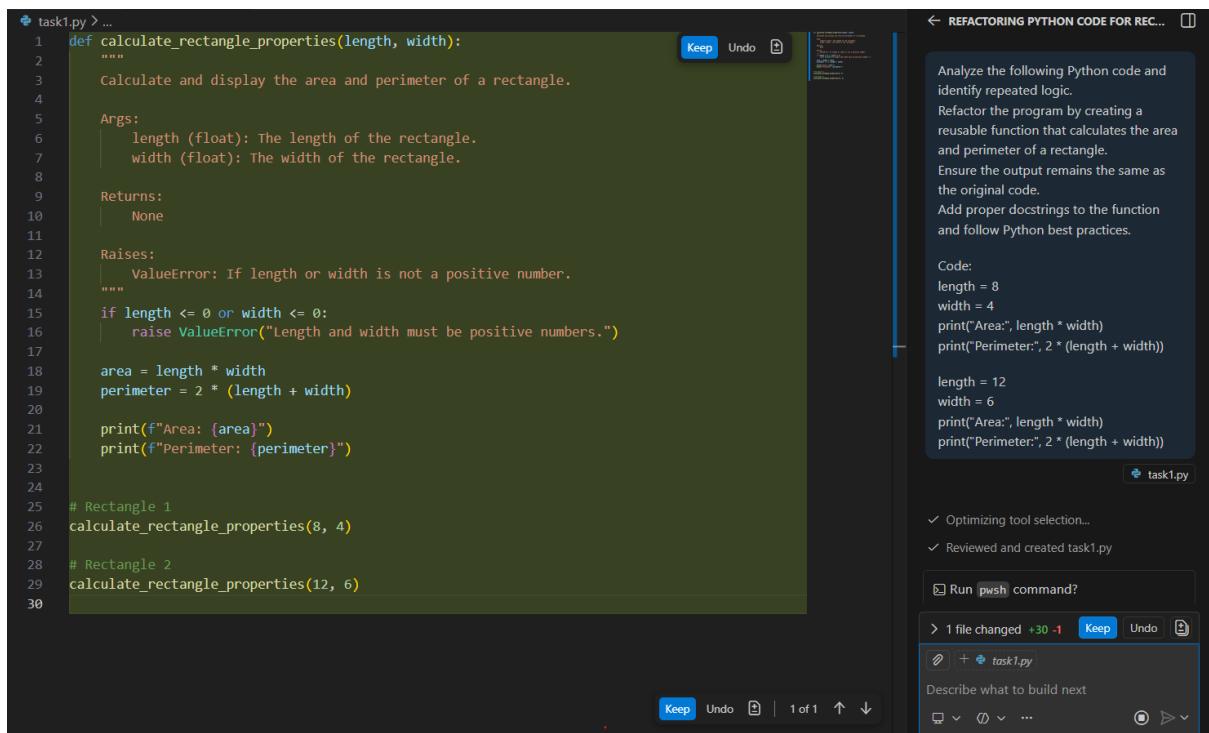
Sample Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.



OUTPUT:

```
Area: 32  
Perimeter: 24  
Area: 72  
Perimeter: 36
```

Task Description -2 (Refactoring – Modularizing Inline Calculations)

• Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

- Identify repeated calculation patterns in the code.
- Convert them into reusable functions.
- Ensure readability and proper documentation.

Sample Code:

```
amount = 1000
```

```
discount = amount * 0.10
```

```
final_amount = amount - discount
```

```
print("Final Amount:", final_amount)
```

```
amount = 2000
```

```
discount = amount * 0.10
```

```
final_amount = amount - discount
```

```
print("Final Amount:", final_amount)
```

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

The screenshot shows a code editor with two panels. The left panel displays the original Python script, and the right panel shows the refactored version with a function.

```
1 def apply_discount(amount, discount_rate=0.10):
2     """
3     Calculate the final amount after applying a discount.
4
5     Args:
6         amount (float): The original amount to apply discount on.
7         discount_rate (float): The discount rate as a decimal (default: 0.10 for 10%).
8
9     Returns:
10        float: The final amount after subtracting the discount.
11    """
12    discount = amount * discount_rate
13    final_amount = amount - discount
14    return final_amount
15
16 # Process first amount
17 amount = 1000
18 final_amount = apply_discount(amount)
19 print("Final Amount:", final_amount)
20
21 # Process second amount
22 amount = 2000
23 final_amount = apply_discount(amount)
24 print("Final Amount:", final_amount)
25
26
```

The right panel shows the refactored code with a function definition and two calls to the function.

```
def apply_discount(amount, discount_rate=0.10):
    """
    Calculate the final amount after applying a discount.

    Args:
        amount (float): The original amount to apply discount on.
        discount_rate (float): The discount rate as a decimal (default: 0.10 for 10%).

    Returns:
        float: The final amount after subtracting the discount.
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount

# Process first amount
amount = 1000
final_amount = apply_discount(amount)
print("Final Amount:", final_amount)

# Process second amount
amount = 2000
final_amount = apply_discount(amount)
print("Final Amount:", final_amount)
```

OUTPUT:

```
Final Amount: 900.0
Final Amount: 1800.0
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

- Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

- Use AI to recommend more efficient data structures or logic simplifications.
- Replace inefficient constructs while preserving program behavior.
- Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]  
check_list = ["Charan", "Esha", "Bhavya"]  
for name in check_list:  
    exists = False  
    for student in students:  
        if name == student:  
            exists = True  
    if exists:  
        print(name, "found")  
    else:  
        print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

```
1  task3.py > ...
2  """
3  PROBLEM ANALYSIS: Inefficient Nested Loop for Name Lookup
4
5  ORIGINAL CODE (INEFFICIENT):
6  - Uses nested loops for lookup (O(n*m) time complexity)
7  - Continues inner loop even after finding a match
8  - Inefficient data structure (list instead of set)
9  """
10 import time
11
12 # =====
13 # 1. ORIGINAL APPROACH (INEFFICIENT)
14 # =====
15
16 def original_approach():
17     """
18     Time Complexity: O(n*m) where n = len(check_list), m = len(students)
19     Space Complexity: O(1)
20
21     Problems:
22     - For each name in check_list (4 iterations)
23     - We iterate through entire students list (4 iterations per check)
24     - Total: 12 operations
25     - No early exit (continues even after finding match)
26     """
27     students = ["Anil", "Bhavya", "Charan", "Divya"]
28     check_list = ["Charan", "Esha", "Bhavya"]
29
30     for name in check_list:
31         exists = False
32         for student in students:
33             if name == student:
34                 exists = True
35         if exists:
36             print(f"{name} found")
37     else:
```

Review the following Python code that uses nested loops to check if student names exist in a list. Identify inefficiencies and suggest an optimized solution using more efficient data structures or logic. Refactor the code to improve lookup performance while preserving the same output. Also compare the efficiency of the original and optimized approach.

Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

task3.py

> 1 file changed +258 -1 Keep Undo

+ task3.py

Describe what to build next

OUTPUT:

```
For this use case, use APPROACH 2 (Set Conversion) because:

✓ Dramatically faster: O(n+m) vs O(n*m)
✓ Simple and readable code
✓ More memory efficient for large datasets
✓ Pythonic and intuitive

Performance gains:
- 1000 students: ~100x faster
- 10000 students: ~1000x faster

Code is also cleaner:
    students_set = set(students)
    if name in students_set:
        print(f"{name} found")

Instead of nested loops!
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

- Detect hardcoded numeric values used directly in expressions.
- Define descriptive constants at the beginning of the program.
- Update calculations to use these constants without changing results.

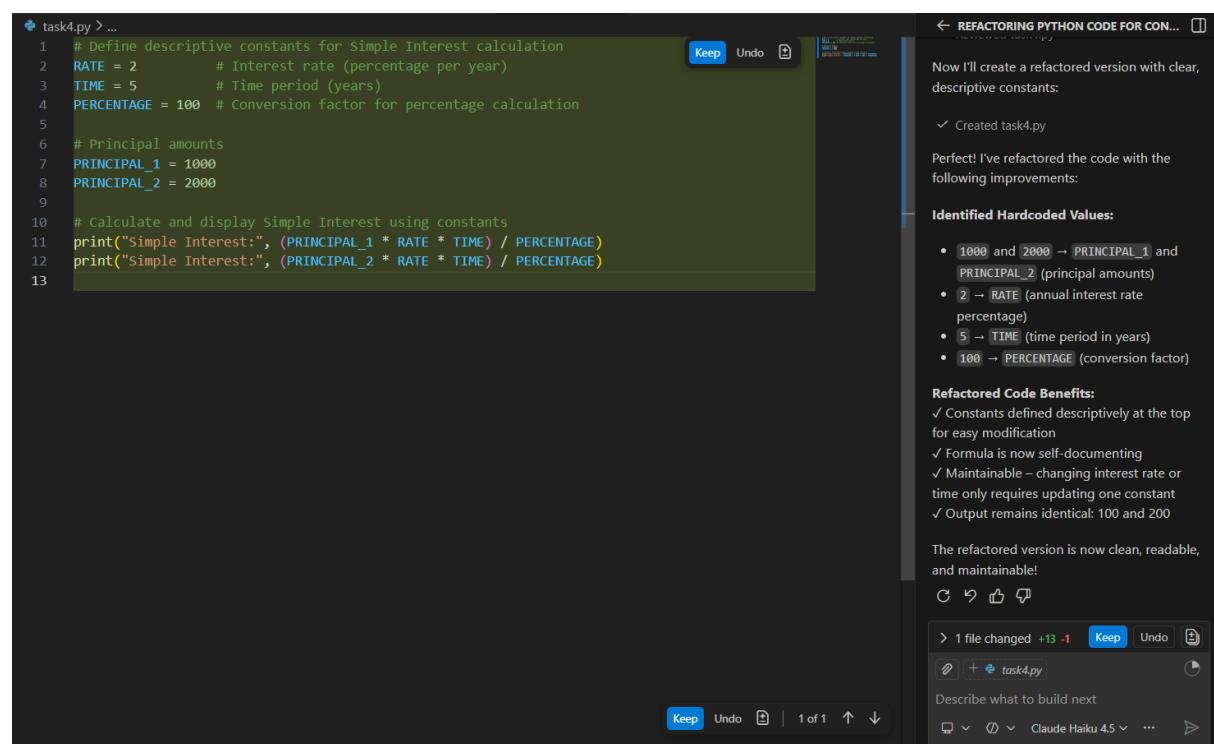
Sample Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
```

```
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations.



```
task4.py > ...
1 # Define descriptive constants for Simple Interest calculation
2 RATE = 2 # Interest rate (percentage per year)
3 TIME = 5 # Time period (years)
4 PERCENTAGE = 100 # Conversion factor for percentage calculation
5
6 # Principal amounts
7 PRINCIPAL_1 = 1000
8 PRINCIPAL_2 = 2000
9
10 # Calculate and display Simple Interest using constants
11 print("Simple Interest:", (PRINCIPAL_1 * RATE * TIME) / PERCENTAGE)
12 print("Simple Interest:", (PRINCIPAL_2 * RATE * TIME) / PERCENTAGE)
13
```

REFACTORING PYTHON CODE FOR CON...

Now I'll create a refactored version with clear, descriptive constants:

- ✓ Created task4.py

Perfect! I've refactored the code with the following improvements:

Identified Hardcoded Values:

- 1000 and 2000 → PRINCIPAL_1 and PRINCIPAL_2 (principal amounts)
- 2 → RATE (annual interest rate percentage)
- 5 → TIME (time period in years)
- 100 → PERCENTAGE (conversion factor)

Refactored Code Benefits:

- ✓ Constants defined descriptively at the top for easy modification
- ✓ Formula is now self-documenting
- ✓ Maintainable – changing interest rate or time only requires updating one constant
- ✓ Output remains identical: 100 and 200

The refactored version is now clean, readable, and maintainable!

> 1 file changed +13 -1 Keep Undo

task4.py

Describe what to build next

Claude Haiku 4.5

OUTPUT:

```
Simple Interest: 100.0
Simple Interest: 200.0
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

- Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

- Replace non-descriptive variable names with meaningful identifiers.
- Add inline comments where the logic may be unclear.
- Ensure program output remains the same.

Sample Code:

```
x = 50
```

```
y = 30
```

```
z = x + y
```

```
print(z)
```

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

task5.py > ...

1 # Initialize the first value

2 first_value = 50

3

4 # Initialize the second value

5 second_value = 30

6

7 # Calculate the sum of both values

8 sum_result = first_value + second_value

9

10 # Display the calculated sum

11 print(sum_result)

12

Keep

Undo

ENHANCING PYTHON CODE READABILITY...

Improve the readability of the following Python code.

Replace unclear variable names with meaningful ones and add explanatory comments where needed.

Do not change the functionality or output of the program.

Code:

x = 50

y = 30

z = x + y

print(z)

task5.py

✓ Reviewed task5.py and considered improving code readability

I'll improve the code readability by replacing the unclear variable names with meaningful ones and adding explanatory comments. Let me first check the current content of task5.py:

✓ Working...

Done! I've improved the code with the following changes:

> 1 file changed +12 -1

Keep

Undo

task5.py

Describe what to build next

Claude Hai...

OUTPUT:

```
80
```